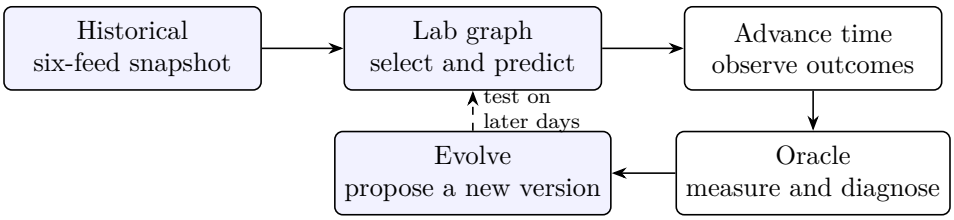
Zenith: learning a financial reasoning graph

Will Cook • 12 October 2026

The idea. Give a reasoning system the information available on a historical trading day. Let it choose what matters and predict what happens next. Compare those predictions with reality. Change the reasoning procedure, then test the changed version on later days. Repeat only while the evidence supports improvement.

The object that learns is a graph of reasoning operators. Each node transforms inputs into outputs. Learning changes its instructions, context, skills or connections; hosted-model weights remain fixed. Test whether the changed procedure makes better later predictions from the same information.



Lab. Run the existing graph. Its operators interpret individual feeds, combine evidence and produce a forecast with an explicit asset, issue time, horizon and numerical target. Save intermediate outputs and the evidence used. Lab may abstain. A smaller selected set is useful only if its selection and predictions improve the declared objective.

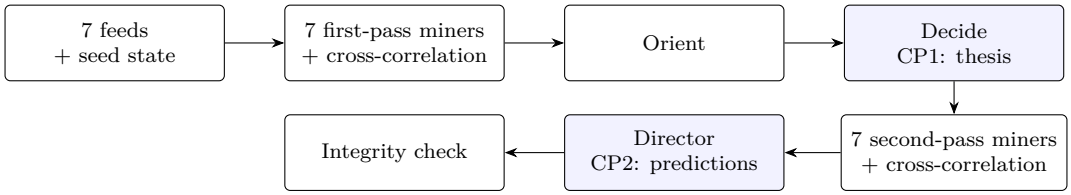
Oracle. Advance the historical clock and measure what happened. Keep two things separate: the observed error and an explanation of that error. A wrong forecast can follow from missing evidence, a misread fact, poor selection, a wrong numerical consequence or unavoidable uncertainty. A later narrative does not identify which cause was responsible.

Evolve. The lead agent, or a worker with a fixed prompt, reads the recorded experience and proposes a specific change. Save the old and new versions. Execute both on the same later inputs before retaining a winner. An attractive explanation earns a test, not automatic adoption.

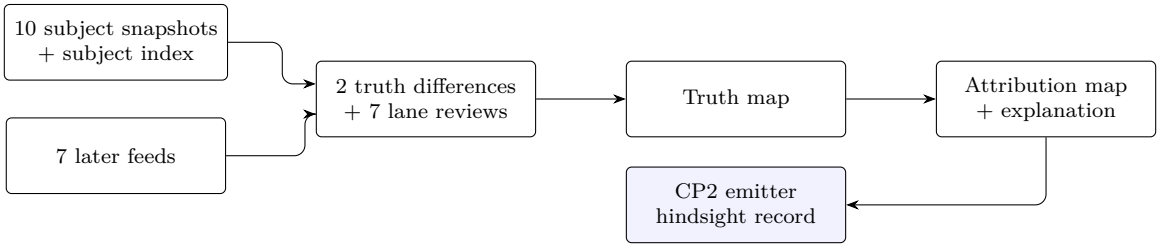
Successive version comparisons draw on AlphaGo Zero [1]; bounded edits and evaluation draw on autoresearch [2]. Finance has no equivalent of generating unlimited games against oneself: market outcomes are external, noisy and limited. Reusing historical outcomes can overfit the research process. The analogy motivates the loop; it supplies neither a learning guarantee nor a performance claim.

The original graphs. These diagrams group parallel nodes to show the reading order. The repository's `docs/graphs.md` shows every actual node and dependency, including the skip connections omitted here.

Lab: 28 nodes, including seven feeds



Oracle: 31 nodes, including seven later feeds



Give these graphs the intended evidence. The original six acquired feeds were stocks, ETFs, macro, Stockgrid, prediction markets and news; the calculator was derived. Recent ten-stock, title-based studies omitted much of that information. Start with raw FNSPID articles and prices [3], join ETF history and archived macro releases, and inspect a few synchronized days. Stockgrid equivalence remains unresolved. Preserve publication clocks and missingness: even original saved news includes post-cutoff items. A richer payload is not automatically a valid historical snapshot.

Measure selection and prediction separately. Freeze the candidate universe U_t before a run. Lab selects $S_t \subseteq U_t$. Report its prediction error and coverage $|S_t|/|U_t|$. To prevent apparent improvement by avoiding difficult cases, also score the whole universe using a fixed fallback forecast for unselected assets:

$$L_t = \frac{1}{|U_t|} \sum_{i \in U_t} \ell\left(y_{it},\ \begin{cases} \hat{y}_{it}, & i \in S_t, \\ b_{it}, & i \notin S_t. \end{cases} \right).$$

Here b_{it} is known at issue time and ℓ is fixed before comparison. If all assets have shadow forecasts, compare the chosen subset with a uniform selection at the same size. This measures selection without turning hindsight into a deployable strategy. Distribution forecasts need a proper score, such as quantile loss [4]. Forecast accuracy and trading returns are separate claims; a trading test also needs an executable timing and cost model.

The smallest scientific loop.

1. Assemble a short sequence of audited daily snapshots with the intended market breadth. Keep absent fields and source limitations visible.
2. Run the existing Lab, save its intermediate outputs and freeze its forecasts. Compare against a simple price baseline and a direct predictor using the same inputs.
3. Reveal matured outcomes. Score the whole declared universe and record validity failures and abstentions. Let Oracle propose explanations.
4. Let Evolve make one understandable candidate change, or a small combined change when components depend on each other. Record why it was proposed.
5. Compare incumbent and candidate on the same later days. Retain a change only under a rule set before those results. Keep failed candidates and repeat.

Input ablations ask whether a feed helps. Procedure comparisons ask whether a reasoning change helps. These are different experiments. Begin with the information question when the current data omits most of what the original graph was designed to interpret.

What exists and what remains unproved. The repository contains the original graph, native Lab and Oracle execution, dated forecast records, outcome scoring and reviewed version changes. A completed two-day development example changed a price rule after feedback; the changed rule then lost to the unchanged rule. Later title-based experiments also failed to establish a predictive improvement. These runs show that the loop can execute, not that it learns a financial edge. Their complete records remain in the research archive and version history.

The next useful result is a small, credible multi-feed historical replay in which the existing graph has the information it needs. Then test whether it interprets that evidence correctly, selects better opportunities and improves its later predictions. If it does not, the recorded intermediate outputs and matched interventions should tell us which change to test next. Synthetic known-signal examples can check this machinery; only financial evidence can support the financial claim.

References

[1] D. Silver and D. Hassabis. *AlphaGo Zero: Starting from scratch*. DeepMind, 2017. https://deepmind.google/blog/alphago-zero-starting-from-scratch/.

[2] A. Karpathy. *autoresearch*. Source inspected at `228791fb499a`, 2026. https://github.com/karpathy/autoresearch.

[3] FNSPID contributors. *Financial News and Stock Price Integration Dataset*. Raw source inspected at `4054842ec476`. https://github.com/Zdong104/FNSPID_Financial_News_Dataset.

[4] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *JASA* 102(477):359–378, 2007. https://doi.org/10.1198/016214506000001437.